

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 4

Развёртывание приложения на удалённом сервере

Выполнил:
Цатинян Артём
Группа БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

1 Задача

Тема проекта: приложение для бронирования столиков в ресторанах.

В рамках лабораторной работы требовалось выполнить ручное развёртывание backend-приложения на удалённом сервере. В качестве развиваемого приложения использовалась микросервисная версия проекта: `identity-service`, `catalog-service`, `booking-service`, `review-service`, PostgreSQL и RabbitMQ.

2 Ход работы

2.1 Подготовка сервера

Для развёртывания был использован удалённый сервер с публичным IP-адресом 186.246.13.204. На сервере установлена операционная система Debian GNU/Linux 13.

На сервер были установлены необходимые компоненты:

```
apt-get update
apt-get install -y git docker.io docker-compose openjdk-21-jdk postgresql
rabbitmq-server
systemctl enable --now docker postgresql rabbitmq-server
```

Docker и Docker Compose были установлены для поддержки контейнерного сценария, однако во время ручного деплоя сервер упёрся в лимиты загрузки образов из публичных registry. Поэтому для финального ручного развёртывания был выбран более стабильный вариант для небольшой VPS: запуск собранных Spring Boot jar-файлов через `systemd`. Такой способ соответствует ручному развёртыванию и не зависит от Docker Hub rate limit.

2.2 Настройка базы данных и брокера сообщений

В PostgreSQL были созданы пользователь и база данных, которые используются сервисами:

```
CREATE ROLE storage LOGIN PASSWORD 'storage';
CREATE DATABASE storage_local OWNER storage;
```

Для RabbitMQ был включён management plugin и создан пользователь:

```
rabbitmq-plugins enable rabbitmq_management
rabbitmqctl add_user storage storage
rabbitmqctl set_user_tags storage administrator
rabbitmqctl set_permissions -p / storage '.*' '.*' '.*'
```

2.3 Сборка приложения

На сервере была предпринята попытка собрать проект через Gradle, но VPS имеет около 2 GB RAM и не имеет swap. Kotlin compiler daemon создавал слишком высокую нагрузку, поэтому сборка была перенесена на локальную машину, а на сервер загружались уже готовые Spring Boot jar-файлы.

Локальная сборка выполнялась командой:

```
./gradlew.bat --console=plain --offline bootJar
```

После сборки на сервер были загружены jar-файлы:

Сервис	Jar-файл
identity-service	identity-service/build/libs/ identity-service.jar
catalog-service	catalog-service/build/libs/ catalog-service.jar
booking-service	booking-service/build/libs/ booking-service.jar
review-service	review-service/build/libs/ review-service.jar

2.4 Настройка systemd

Для запуска микросервисов был добавлен скрипт `deploy-systemd.sh`. Он создаёт четыре unit-файла:

Unit	Порт сервиса
restaurant-booking-identity.service	8081
restaurant-booking-catalog.service	8082
restaurant-booking-booking.service	8083
restaurant-booking-review.service	8084

Для каждого сервиса задаются переменные окружения подключения к PostgreSQL и RabbitMQ. Пример unit-файла:

```
[Service]
WorkingDirectory=/opt/restaurant-booking-lab2
Environment=SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/  
storage_local
Environment=SPRING_DATASOURCE_USERNAME=storage
Environment=SPRING_DATASOURCE_PASSWORD=storage
Environment=SPRING_RABBITMQ_HOST=localhost
```

```
ExecStart=/usr/bin/java -jar /opt/restaurant-booking-lab2/identity-service/  
    build/libs/identity-service.jar  
Restart=always
```

Сервисы включены в автозагрузку:

```
systemctl enable restaurant-booking-identity  
systemctl enable restaurant-booking-catalog  
systemctl enable restaurant-booking-booking  
systemctl enable restaurant-booking-review
```

2.5 Результат развёртывания

После запуска все четыре сервиса находятся в состоянии **active**. Проверка выполнялась командами:

```
systemctl is-active restaurant-booking-identity restaurant-booking-catalog  
    restaurant-booking-booking restaurant-booking-review  
curl http://localhost:8081/v3/api-docs  
curl http://localhost:8082/v3/api-docs  
curl http://localhost:8083/v3/api-docs  
curl http://localhost:8084/v3/api-docs
```

Также была проверена пользовательская ручка каталога:

```
curl http://localhost:8082/api/v1/cuisines
```

Ответ содержал тестовые кухни:

```
[  
  {"id":1,"name":"European"},  
  {"id":2,"name":"Italian"},  
  {"id":3,"name":"Japanese"},  
  {"id":4,"name":"Vegetarian"}  
]
```

2.6 Настройка nginx reverse proxy

По полному условию лабораторной работы также был настроен nginx в режиме обратного прокси. На сервер уже был направлен домен **ezhidze.ru**, поэтому приложение было опубликовано через HTTPS на этом домене. Старый активный конфиг сайта был сохранён в резервную копию, а доменный конфиг nginx заменён на reverse proxy для микросервисов.

Корневой маршрут перенаправляет пользователя на Swagger каталога:

```
location = / {  
    return 302 /catalog/swagger-ui/index.html;  
}
```

Для сервисов настроены отдельные префиксы:

```
location /identity/ { proxy_pass http://127.0.0.1:8081/; }
location /catalog/ { proxy_pass http://127.0.0.1:8082/; }
location /booking/ { proxy_pass http://127.0.0.1:8083/; }
location /review/ { proxy_pass http://127.0.0.1:8084/; }
```

После настройки выполнялась проверка:

```
nginx -t
systemctl reload nginx
curl https://ezhidze.ru/catalog/api/v1/cuisines
curl https://ezhidze.ru/identity/v3/api-docs
curl https://ezhidze.ru/catalog/v3/api-docs
curl https://ezhidze.ru/booking/v3/api-docs
curl https://ezhidze.ru/review/v3/api-docs
```

После развёртывания приложение доступно через nginx:

Сервис	Swagger UI
identity-service	https://ezhidze.ru/identity/swagger-ui/index.html
catalog-service	https://ezhidze.ru/catalog/swagger-ui/index.html
booking-service	https://ezhidze.ru/booking/swagger-ui/index.html
review-service	https://ezhidze.ru/review/swagger-ui/index.html

Прямые порты сервисов 8081-8084 также остаются доступными для отладки, но основной внешний доступ для защиты организован через nginx и домен `ezhidze.ru`.

3 Вывод

В рамках лабораторной работы микросервисное backend-приложение было вручную развёрнуто на удалённом сервере. На сервере установлены Java 21, PostgreSQL, RabbitMQ и nginx, настроены база данных и брокер сообщений, а четыре Spring Boot сервиса запущены как systemd-сервисы с автоперезапуском и автозагрузкой.

Результат проверен через Swagger/OpenAPI endpoints и тестовую ручку каталога. Приложение доступно извне через nginx reverse proxy по домену `ezhidze.ru`.